

Automated Maze Solver: Progress Report & Adjusted Strategy

Dipra

September 26, 2026

1 Introduction

With only two days remaining until the project submission deadline, this report outlines the significant progress made on the Automated Maze Solver, details the recurring hardware and software challenges encountered, and presents a revised, focused strategy to ensure a successful and complete demonstration.

2 Progress Achieved

Considerable milestones have been reached in developing the dual-microcontroller architecture of the maze solver:

- **Dual-MCU Architecture:** Successfully established a robust master-slave configuration. The ESP32 acts as the high-level brain handling sensors and navigation algorithms, while the ATmega32 acts as the motor controller, receiving commands via UART.
- **Sensor Integration:** Successfully interfaced three VL53L0X Time-of-Flight (ToF) sensors over a shared I2C bus using XSHUT pins for dynamic address assignment.
- **Gyroscope Tracking:** Integrated the MPU6050 for heading tracking. Implemented a Proportional-Derivative (PD) control loop allowing the car to maintain a straight trajectory and dynamically correct for motor imbalances.
- **Telemetry & Debugging:** Built a comprehensive Bluetooth telemetry system that mirrors internal ESP32 logs to a mobile device, allowing for real-time debugging without physical cable tethers.
- **Fault Tolerance:** Implemented a software fallback mechanism (Timed Turns) and I2C bus recovery sequence (9-clock cycle SCL pulse) to recover from mid-operation hardware freezes.

3 Major Recurring Challenges

The project has faced persistent challenges, predominantly stemming from hardware and electrical limitations:

- **Electrical Noise and I2C Bus Instability:** The most significant hurdle has been electrical noise generated by the brushed DC motors. During sudden power draws (especially during pivoting turns), voltage spikes/drops cause the sensitive I2C bus to crash, leading to "Error 263" and freezing the Gyroscope (MPU6050) and ToF sensors.
- **Motor Asymmetry:** The right and left motors exhibit different physical torque characteristics, causing the car to naturally drift. While the PD controller attempts to compensate, extreme corrections have sometimes led to runaway oscillations.
- **Sensor Lockups During Turns:** Because the I2C bus freezes during aggressive turns, the closed-loop gyro turning algorithm loses its feedback. We successfully implemented a software fallback (open-loop timed turns), but physical calibration remains sensitive to battery voltage.

4 Revised Strategy and Reduced Scope

Given the hard two-day deadline, the project scope must be tightened to guarantee a functional demonstration rather than chasing perfect hardware reliability. The strategy is now focused on demonstrating the core academic concepts: **memory efficiency** and **branching logic**.

4.1 Demonstration Goals

1. **Memory Saving (ATmega32):** Highlight the dual-MCU architecture as a memory-saving strategy. Show how offloading the heavy floating-point PID math, Bluetooth stack, and I2C sensor libraries to the ESP32 allows the ATmega32 to run a lightweight, highly responsive motor driver loop.
2. **Branching Logic & Navigation:** Construct a simplified maze environment containing distinct branching points (e.g., T-junctions or Dead Ends). Demonstrate the ESP32 successfully reading the ToF sensors to map the intersection and making an autonomous decision to navigate through the branch.

4.2 Execution Plan

- **Embrace the Fallback:** Instead of spending hours attempting to hardware-filter the I2C electrical noise with capacitors, we will heavily rely on the **Timed Turn** fallback. We will manually tune the `ms_per_deg` constant to execute a roughly accurate 90-degree turn.
- **Conservative Speeds:** Limit the base PWM speeds and Kp values to prevent aggressive maneuvers that trigger electrical brownouts.
- **Simple Maze Layout:** Use a short track with clear, straight walls and one or two well-defined decision points rather than a massive, complex labyrinth.

5 Conclusion

The underlying logic and architecture of the maze solver are fundamentally sound and sophisticated. By shifting the focus away from perfect hardware execution towards a conceptual demonstration of dual-MCU memory distribution and intersection branching, the project can be successfully presented within the remaining timeframe.